Programmierhandbuch
tails-pdp
Architektur, Schnittstellen und Erweiterung

Christian Koch

Stand: 25. Oktober 2026

Inhaltsverzeichnis

1 Zweck und Quellstand **1**

2 Workspace und Verantwortlichkeiten **1**

3 Kontrollfluss und Entscheidungslogik **1**
3.1 Initialisierung und Lebenszyklus 1
3.2 Neue Dateiöffnung 2
3.3 Nachbewertung offener Deskriptoren 2

4 Maps und binäre Schnittstellen **3**

5 Dateibasierte Konfigurationsschnittstelle **3**
5.1 Policyformat 3
5.2 Attributformat 4
5.3 Veröffentlichung und Fehlerverhalten 5

6 Rust-Schnittstellen und Fehlerkontrakte **5**
6.1 Policyloader 5
6.2 Attribut- und Zeitloader 6
6.3 Trigger und Userspace-PEP 6
6.4 Gemeinsame Policylogik 7

7 Admintool und Diagnoseaufrufe **7**

8 Build, Tests und Erweiterung **8**
8.1 Entwicklungsbefehle 8
8.2 Geführte Demo und Isolation 8
8.3 API-Dokumentation mit Rustdoc 9
8.4 Testzuordnung 9
8.5 Erweiterungsschritte 10

9 Grenzen und Nachweiszuordnung **10**

10 Ausgabe des Handbuchs **11**

1 Zweck und Quellstand

Dieses Handbuch beschreibt die Programmierung und Wartung des ASBAC-Prototyps `tails-pdp`. Grundlage ist der bereinigte Workspace mit Skripten unter `scripts/` und ohne die stillgelegten Socket-Bind-Datentypen. Die Installation auf dem Linux-Zielsystem ist im separaten Installationshandbuch beschrieben. Die Dokumente sind unabhängig voneinander kompilierbar.

Die Runtime unterstützt `file_open`: Neue Dateiöffnungen werden über BPF-LSM entschieden. Ein Userspace-PEP bewertet bereits offene Dateideskriptoren bei relevanten Änderungen nach. Policies und Attribute werden aus Dateien geladen; das Admintool ist eine lesende Diagnosekomponente. Der Parser unterstützt einen beschränkten SAPL-inspirierten Ausschnitt, keine vollständige SAPL-Engine.

2 Workspace und Verantwortlichkeiten

Alle Pfade beziehen sich, sofern nicht anders angegeben, auf die Repository-Wurzel.

Crate bzw. Verzeichnis	Aufgabe
`tails-pdp/`	Userspace-Einstieg, eBPF-Laden, Initialisierung, Attach und Lebenszyklus der asynchronen Aufgaben.
`tails-pdp-ebpf/`	LSM-Hook, Tail-Call-Kette, Maps und Auswertung neuer Dateiöffnungen; `no_std`.
`tails-pdp-common/`	Gemeinsame Datentypen, Map-Layouts, Konstanten und Policy-/Attributlogik; `no_std`, optionale Userspace-Funktionen über Feature `user`.
`tails-pdp-policy-loader/`	Dateiparser, Übersetzung und generationenbasierte Aktivierung der Policies.
`tails-pdp-attribute-loader/`	Zeitmap, strukturierte Attribute und Aktivierung neuer Attributgenerationen.
`tails-pdp-userspace-common/`	Gepinnte Maps, Aktivierungstrigger, rekursiver Dateiwatcher und Messmarkierungen.
`tails-pdp-userspace-pep/`	Nachbewertung offener FDs und versuchter Entzug verletzender Deskriptoren.
`tails-pdp-admintool/`	Kommandozeile und lesende Ausgabe von Policies, Attributen und aktivem Zustand.
`scripts/tests/`	Komponenten-, E2E-, Performance-, Last-, Stabilitäts- und Demo-Regressionsprüfungen einschließlich Pythonhelfern.
`scripts/demo/`	Geführte tmux-Demo, Offline-Buildhelfer, Werkzeugprüfung und isolierter Demoablauf.
`examples/`	Inaktive Policy- und Attributbeispiele zur gezielten Veröffentlichung.

3 Kontrollfluss und Entscheidungslogik

3.1 Initialisierung und Lebenszyklus

`tails-pdp/src/main.rs` richtet das Pin-Verzeichnis ein und prüft vorhandene Map-Layouts. Es lädt das eingebettete eBPF-Objekt und die BTF-Daten, lädt die LSM-Programme und belegt die Tail-Call-Tabelle. Danach werden Policies, Zeit und Attribute

initial geschrieben. Erst anschließend wird der Einstiegshook angehängt.

Nach dem Attach laufen Zeitpflege, Attributwatcher, Policysynchronisierung und Userspace-PEP über `tokio::select!`. Strg+C beendet die Runtime; die Rückkehr einer dieser Aufgaben beendet ebenfalls den gemeinsamen Ablauf und propagiert gegebenenfalls den Fehler. Es gibt keinen separat betriebenen Policy-Daemon und keine HTTP-Schnittstelle. Gepinnte Maps können das Prozessende überdauern.

3.2 Neue Dateiöffnung

Der Hook `file_open` erhält Requestdaten aus dem Kernel. Die Ressource wird über Gerät und Inode identifiziert, der Prozess über UID und Kernel-Commandnamen. Ein bereits negativer Rückgabewert einer vorausgehenden BPF-LSM-Entscheidung wird erhalten. Danach folgt die Kette:

```
file_open
  -> evaluate_file_open_static_policies  (Tail-Index 0)
  -> evaluate_file_open_stream_policies  (Tail-Index 1)
  -> combine_file_open                  (Tail-Index 2)
```

Die Jump-Table-Indizes und Programmnamen sind in `tails-pdp/src/lib.rs` und `tails-pdp-ebpf/src/maps.rs` aufeinander abgestimmt. Ein fehlgeschlagener Tail-Call oder fehlender Entscheidungszustand führt zur Ablehnung. Die Kombination verwendet Deny-Overrides: Eine passende Deny-Policy führt zu `-EPERM`; ohne passende Deny-Policy ist die Entscheidung erlaubend. Eine Permit-Policy überstimmt kein Deny. Ein leerer Policystand erlaubt daher Dateiöffnungen, sofern die übrigen Sicherheitsmechanismen sie erlauben.

3.3 Nachbewertung offener Deskriptoren

`tails-pdp-userspace-pep/src/pep.rs` empfängt Aktivierungstrigger und plant zusätzlich Nachbewertungen an relevanten Zeitbedingungsgrenzen. Es scannt `/proc` nicht periodisch unabhängig von diesen Ursachen. Ein Scan verwendet einen gemeinsamen Kontext aus Policygeneration, Attributgeneration, Attributbank und Zeit. Prozess- und FD-Daten werden auf Dateiidentitäten zurückgeführt und gegen die Policylogik geprüft.

Der Entzug wird über `tails-pdp-userspace-pep/src/fd_revoker.rs` versucht. Unter x86_64-Linux wird der Zielprozess mit `ptrace` angehalten; ein `close`-Systemaufruf wird injiziert und Code-/Registerzustand werden wiederhergestellt. Auf anderen Plattformen liefert die Entzugsimplementierung einen ausdrücklichen Fehler. PID 0, PID 1 und der Runtime-Prozess sind geschützt. Der Ablauf ist nicht atomar mit allen konkurrierenden Prozessoperationen.

4 Maps und binäre Schnittstellen

Die Maps werden in `tails-pdp-ebpf/src/maps.rs` definiert. Gepinnte Maps liegen unter `/sys/fs/bpf/tails-pdp/`.

Map	Inhalt und Kapazität
`FILE_OPEN_STATIC_POLICIES`	Array mit 32 `FileOpenStaticPolicy`-Einträgen: zwei Bänke mit je 16 Policies.
`FILE_OPEN_STREAM_POLICIES`	Array mit 32 `FileOpenStreamPolicy`-Einträgen: zwei Bänke mit je 16 Policies.
`POLICY_GENERATION`	Array mit einer aktiven Generationsnummer vom Typ `u32`.
`ATTRIBUTES`	HashMap mit höchstens 1024 Einträgen; Schlüssel `AttributeKey`, Wert `AttributeValue`.
`ATTRIBUTE_GENERATION`	Array mit einer aktiven Attributgeneration vom Typ `u32`.
`CURRENT_TIME`	Array mit einer Unix-Zeit in Sekunden vom Typ `u64`.
`FILE_OPEN_JUMP_TABLE`	Nicht gepinnte ProgramArray mit drei Tail-Call-Zielen.
`DECISIONS`	Nicht gepinnte PerCpuArray mit Deny-, Permit- und Policygenerationszustand.
`DEBUG_LOGGING`	Nicht gepinnte Array mit einem Debug-Schalter.

Die gemeinsamen `repr(C)`-Strukturen und expliziten Paddingfelder bilden die Schnittstelle zwischen Kernel und Userspace. `repr(u8)`-Enums haben explizite Werte; `PolicyAction::FileOpen` bleibt `1`. Die File-Open-Policystrukturen besitzen im geprüften Layout 104 bzw. 320 Byte bei Ausrichtung 8. Direkte Map-Zugriffe erfordern exakt passende Typen und Größen. Padding, Feldreihenfolge und Enumwerte dürfen nicht beiläufig verändert werden. `aya::Pod`-Implementierungen sind Teil dieser Verantwortung.

Die Policybank beginnt bei `(generation % 2) * 16`; die Attributbank wird durch `generation % 2` gewählt. Loader schreiben zunächst den inaktiven Stand und schalten danach die Generation um. Policy- und Attributgeneration sind getrennte Zustände; mehrere veröffentlichte Dateien und die beiden Loader sind keine gemeinsame Transaktion. Die Attributkapazitätsprüfung berücksichtigt die tatsächlich gepinnte Map und den noch benötigten aktiven Bestand. Eine feste Garantie von 512 Attributen je Generation lässt sich daraus nicht ableiten.

5 Dateibasierte Konfigurationsschnittstelle

5.1 Policyformat

Unter `policies/` einschließlich Unterverzeichnissen wird jede Datei mit Endung `.policy` als genau ein Policydokument gelesen. Der Pfad ergibt sich aus dem Arbeitsverzeichnis der Runtime, nicht dem Ort des Programmbinaries.

```
policy "deny demo for engineers"
deny
    action == "file_open";
```

```
    command == "cat";
    resource.path == "/tmp/tails-pdp-programmer-demo.txt";
    subject.position == "engineer";
    system.defcon <= 3;
```

Die Entitlement-Zeile ist `permit` oder `deny`. Statements enden mit Semikolon und werden als Konjunktion verknüpft. Unterstützt sind:

- `action == file_open"` (auch Schreibweise `file-open`);
- optionale statische Filter `subject.uid`, `command` und `resource.path`;
- Zeitbedingungen wie `environment.time % N < VALUE` oder Vergleiche mit `environment.utc.hour`, `minute` und `second`;
- bis zu vier dynamische Bedingungen auf `system.*`, `subject.*` und `resource.*` sowie höchstens eine zusätzliche Zeitbedingung;
- numerische Vergleiche <, <=, ==, >=, >; Stringattribute werden nur mit Gleichheit verglichen.

Nicht unterstützt sind unter anderem Socket-Bind-Aktionen, Imports, Schemata, Variablen, Obligations, Advice, Transformationen und `or`. Disjunktionen können für passende Anwendungsfälle als mehrere Policies modelliert werden; dabei gilt weiterhin Deny-Overrides.

Die maximale Commandlänge beträgt 15 Byte; Linux reserviert im 16-Byte-Commandfeld ein Nullbyte. Ressourcenpfade sind auf 64 Byte begrenzt. Der Loader löst vorhandene Ressourcenpfade in Gerät/Inode auf. Eine später am selben Pfad ersetzte Datei besitzt möglicherweise eine andere Identität; die Regel ist keine laufende Pfadnamensüberwachung.

5.2 Attributformat

Die Dateien verwenden die Endung `.attributes` und Einträge `name = wert`. Beispiele:

```
# attributes/system.attributes
defcon = 3

# attributes/subjects/1000.attributes
position = "engineer"
access-level = 2
on_call = true

# attributes/resources/tmp/tails-pdp-programmer-demo.txt.attributes
classification = "internal"
clearanceLevel = 3
```

Die drei Namensräume sind System, Subject und Resource. Subjectdateien werden über die UID benannt. Resource-Dateipfade bilden den absoluten Ressourcenpfad ohne führenden Schrägstrich plus Attributsuffix ab; beim Laden wird die Identität der beschriebenen Datei

aufgelöst. Globale Systemattribute haben keine requestabhängige Objektidentität. Namen und Stringwerte werden durch gemeinsame Hashfunktionen in die Map-Repräsentation überführt. Zahlen sind vorzeichenlose Ganzzahlen, boolesche Werte sind `true`/`false`; DEFCON muss zwischen 1 und 5 liegen. Regeln mit `subject.position` benötigen die Attributdatei für die tatsächlich anfragende UID, nicht zwingend für UID 1000.

5.3 Veröffentlichung und Fehlerverhalten

`scripts/publish-config.sh` erhält genau zwei Argumente: eine vollständig vorbereitete Quelldatei und das Ziel mit Endung `.policy` oder `.attributes`. Es schreibt eine temporäre Datei im Zielverzeichnis und ersetzt das Ziel durch Umbenennung auf demselben Dateisystem. Bei Ersetzung werden die Zugriffsbits des vorhandenen Ziels erhalten; Eigentümer ist der veröffentlichende Prozess. Symlink- und Verzeichnisziele werden abgelehnt.

```
printf 'demo\n' > /tmp/tails-pdp-programmer-demo.txt
bash scripts/publish-config.sh /tmp/prepared.policy policies/demo.policy
bash scripts/publish-config.sh /tmp/prepared.attributes attributes/system.
    attributes
rm policies/demo.policy
```

Die vorbereiteten Quelldateien müssen vorher mit vollständigen gültigen Inhalten erstellt werden. Der rekursive Inotify-Watcher reagiert auf Veröffentlichung und Entfernung; temporäre Schreibvorgänge werden ignoriert. Es gibt keinen festen Debounce-Timer. Die Veröffentlichung validiert die Policysemantik nicht und bestätigt nicht die spätere Aktivierung. Laufende ungültige Updates erhalten den letzten gültigen Stand; initial ungültige Daten können den Start verhindern. Bestätigung erfolgt über Runtime-Log und Admintool.

6 Rust-Schnittstellen und Fehlerkontrakte

Die folgenden Schnittstellen werden vom Runtime-Einstieg verwendet. Sie ersetzen keine vollständige Rustdoc-Auflistung, beschreiben aber die zentralen Integrationspunkte.

6.1 Policyloader

```
PolicyDirectorySync::new(
    enforcement_triggers: mpsc::Sender<EnforcementTrigger>
) -> anyhow::Result<Self>

PolicyDirectorySync::directory(&self) -> &Path
PolicyDirectorySync::sync_initial(&mut self) -> anyhow::Result<()>
PolicyDirectorySync::run(self) -> impl Future<Output = anyhow::Result<()>>
verify_pinned_map_layouts() -> anyhow::Result<()>
```

new erstellt das Standardverzeichnis und öffnet die benötigten gepinnten Maps. sync_initial validiert und aktiviert den vollständigen initialen Stand; Fehler werden zurückgegeben. run ist im Quellcode eine async fn, konsumiert das Objekt und verarbeitet Verzeichnisänderungen. Ein Aktivierungstrigger wird erst nach erfolgreichem Commit ausgelöst. Die Layoutprüfung erfolgt vor dem Laden des eBPF-Objekts.

6.2 Attribut- und Zeitloader

```
open_attribute_maps() -> anyhow::Result<AttributeMaps>
write_current_attributes(
    attribute_maps: &mut AttributeMaps,
    enforcement_triggers: &mpsc::Sender<EnforcementTrigger>
) -> anyhow::Result<()>
async fn run_attribute_updater(
    attribute_maps: &mut AttributeMaps,
    enforcement_triggers: mpsc::Sender<EnforcementTrigger>
) -> anyhow::Result<()>

open_current_time_map() -> anyhow::Result<Array<MapData, u64>>
write_current_time(map: &mut Array<MapData, u64>) -> anyhow::Result<()>
async fn run_current_time_updater(
    map: &mut Array<MapData, u64>
) -> anyhow::Result<()>
```

AttributeMaps kapselt Attribute und Generationsmap. Der initiale Attributschreibvorgang lädt die Dateien und aktiviert eine Generation. Der Updater validiert neue Inhalte und Kapazitäten vor der Aktivierung. Die Zeitpflege schreibt Unix-Sekunden in CURRENT_TIME[0] und aktualisiert sie im Ein-Sekunden-Takt. UTC-Komponenten werden durch PolicyTime abgeleitet.

6.3 Trigger und Userspace-PEP

```
enum EnforcementTrigger {
    PolicyGenerationActivated { generation: u32 },
    AttributeGenerationActivated { generation: u32 },
    TimeConditionChanged { unix_seconds: u64 },
}

notify_enforcement(
    sender: &mpsc::Sender<EnforcementTrigger>,
    trigger: EnforcementTrigger
) -> anyhow::Result<bool>

async fn run_userspace_pep(
    enforcement_triggers: mpsc::Receiver<EnforcementTrigger>
) -> anyhow::Result<()>
```

Der Kanal hat Kapazität 1. notify_enforcement verwendet try_send: true bedeutet eingereiht; ein voller Kanal liefert false, weil eine Nachbewertung bereits aussteht. Ein geschlossener Kanal ist ein Fehler. Trigger sind keine verlustfreie Historie aller Generationen. Zeittrigger werden vom PEP beim Erreichen einer berechneten Zeitgrenze intern gebildet; sie enthalten deren Unix-Sekunden. Der PEP bewertet den aktuellen aktiven Zustand. Der FD-Entzugshelfer ist ein internes Modul und wird nicht als öffentliche Crate-Schnittstelle exportiert.

6.4 Gemeinsame Policylogik

tails-pdp-common/src/lib.rs enthält unter anderem FileOpenRequest, FileOpenStaticPolicy, FileOpenStreamPolicy, AttributeCondition, AttributeKey, AttributeValue und PolicyTime. evaluate_file_open_static_policy liefert ein optionales Entitlement; None bezeichnet eine nicht passende Policy. Die Stream-Auswertung trennt statische Requestfilter, Zeitbedingung und dynamische Map-Abfragen. Die noch verwendeten Funktionen mit legacy im Namen verarbeiten aktive Zeitbedingungen und dürfen nicht allein wegen ihrer Benennung entfernt werden.

7 Admintool und Diagnoseaufrufe

Das Admintool öffnet Maps lesend. Es bietet show, show-active, show-policies und show-attributes; set oder set-stream existieren nicht.

```
sudo ./target/release/tails-pdp-admintool show-active
sudo ./target/release/tails-pdp-admintool show-policies
sudo ./target/release/tails-pdp-admintool show-attributes
./target/release/tails-pdp-admintool --help
```

Root ist in den Beispielen für den Map-Zugriff vorgesehen; tatsächlich ausreichende Leserechte können auch ohne sudo genügen. Optionen wie -policy-dir und -attributes-dir dienen der Zuordnung der Quellverzeichnisse bei der Anzeige; sie ändern nicht die Verzeichnisse der Runtime. Map-Pfade lassen sich über die im CLI definierten Pin-Pfadoptionen auswählen.

```
sudo env RUST_LOG=debug ./target/release/tails-pdp
sudo env TAILS_PDP_EBPF_DEBUG=1 RUST_LOG=info ./target/release/tails-pdp
sudo cat /sys/kernel/debug/tracing/trace_pipe
sudo env TAILS_PDP_TIMING=1 ./target/release/tails-pdp
```

RUST_LOG steuert Userspace-Logs; Standard ist info. TAILS_PDP_EBPF_DEBUG aktiviert Kernel-Debugausgaben. Der Trace-Aufruf benötigt eine verfügbare Tracefs-/Debugfs-Ablage. TAILS_PDP_TIMING=1 aktiviert opt-in Messmarkierungen auf Basis der Linux-Monotonzeit. Logs und Marker beeinflussen Performance-Messungen und müssen bei deren Interpretation berücksichtigt werden.

8 Build, Tests und Erweiterung

8.1 Entwicklungsbefehle

Buildwerkzeuge und Linux-Voraussetzungen sind im Installationshandbuch beschrieben. `nix-config/rust-toolchain` legt aktuell Rust `1.93.0` für Userspace fest; `nix-config/ebpf-toolchain` legt `nightly-2026-01-15` für den eBPF-Build fest. `tails-pdp/build.rs` liest die eBPF-Version und übergibt sie an Aya. Die Nix-Shell wählt die Userspace-Version und unterbindet implizite Rustup-Installationen. Der aktuelle Aya-eBPF-Build setzt weiterhin Nightly und dessen `rust-src` voraus.

Die Quellenersetzung unter `.cargo/config.toml` und der Offline-Modus sind aktiv. `vendor/` enthält die Projektabhängigkeiten und die unter `nix-config/ebpf-sysroot/` beschriebenen Registry-Pakete des Systembibliotheksbuilds. `scripts/check-vendor.py` prüft deren Vollständigkeit und Prüfsummen; der Buildhelfer vergleicht auch das Lockfile der installierten Rust-Quellen. Compiler, LLVM, Linker und Nix-Store-Abhängigkeiten sind gesonderte Voraussetzungen.

Innerhalb der vorbereiteten Nix-Shell:

```
bash scripts/demo/build.sh
cargo fmt --all -- --check
cargo test --locked -p tails-pdp-common -p tails-pdp-policy-loader --config '
    target."cfg(all())".runner="env"'
cargo doc --locked --no-deps -p tails-pdp-common -p tails-pdp-policy-loader
```

Der Buildhelfer verwendet `cargo build -frozen` und prüft vorhandene Toolchains, Rust-Quellen und Linker. Er installiert keine fehlenden Werkzeuge. Die Test-Runner-Überschreibung verhindert Root-Ausführung normaler Rust-Tests; das Workspace-Cargo-Setup verwendet sonst `sudo -E`. Der Hauptbuild baut über `tails-pdp/build.rs` auch das eBPF-Objekt. Die Buildskripte verbleiben in ihren Crates. Generierte Kerneltypen werden nur gezielt untersucht und nicht pauschal verändert.

8.2 Geführte Demo und Isolation

`scripts/demo.sh` ist der Einstieg für die Demonstration. Der Launcher richtet tmux ein und startet den privilegierten Worker. Der fachliche Ablauf liegt in `scripts/demo/runner.sh`, die System-sudo-Auswahl in `scripts/demo/dependencies.sh`, der Offline-Build in `scripts/demo/build.sh` und die Terminaldarstellung in `scripts/demo/pane.sh`.

```
./scripts/demo.sh
./scripts/demo.sh --auto
./scripts/demo.sh --build
TAILS_PDP_BIN=/absoluter/pfad/tails-pdp ADM_TOOL_BIN=/absoluter/pfad/tails-pdp-
    admintool ./scripts/demo.sh
```

Die Optionen steuern Eingabepausen beziehungsweise einen erzwungenen Neubuild. Ohne Binaries wird automatisch ein Build angefordert. Fehlende Werkzeuge führen entweder zur lokalen Nix-Demo-Shell oder zu einer Fehlermeldung; Nix-Downloads und Nix-Builds sind für diese Bereitstellung deaktiviert. Die vollständige Offline-Werkzeugumgebung ist derzeit nicht im Repository enthalten.

Ein privater Mount-Namespace mit eigenem bpffs erhält die Host-Pins. Die Demo sichert vorhandene Konfigurationsverzeichnisse und `demo/files/` und stellt sie beim regulären Aufräumen wieder her. Logs bleiben unter `demo/runs/`. LSM-Hooks wirken weiterhin systemweit: keine zweite Runtime und keine parallele Evaluation starten. `DEMO_TIMEOUT_SECONDS` begrenzt technische Wartephasen standardmäßig auf 30 Sekunden. Weitere Optionen dokumentiert `scripts/demo/README.md`.

8.3 API-Dokumentation mit Rustdoc

Die Crates und ihre öffentlichen Schnittstellen sind unmittelbar im Quelltext mit `//!`- bzw. `///`-Kommentaren dokumentiert. Die Referenz erläutert Parametersemantik, Rückgaben, Fehlerfälle, Wildcards, Datenlayouts und relevante Plattformgrenzen. Wesentliche interne Algorithmen sind zusätzlich an den betreffenden Routinen dokumentiert. Signaturen und Typverweise erzeugt Rustdoc aus dem Code.

Auf dem vorbereiteten Linux-Zielsystem erzeugt ein gemeinsamer Einstieg die Referenz aller acht Bibliotheksziele:

```
./scripts/docs.sh
./scripts/docs.sh --document-private-items
cargo test --locked --doc -p tails-pdp-common --config 'target."cfg(all())".
    runner="env"'
```

Das Skript prüft fehlende öffentliche Dokumentation und ungültige Rustdoc-Verweise als Fehler. Die zweite Variante enthält zusätzlich interne Symbole. Die HTML-Ausgabe liegt unter `target/doc/`; bei einem expliziten Cross-Target unter `target/<target>/doc/`. Die gesamte HTML-Ablage einschließlich Ressourcen kann separat in das Abgabepaket kopiert werden. Sie wird nicht als Teil des Buildverzeichnisses versioniert. Der Beispieltest benötigt keine BPF-Pins und keine Root-Rechte; privilegierte Runtime-Tests bleiben ein eigener Nachweis.

8.4 Testzuordnung

```
bash scripts/tests/test-publish-config.sh
python3 -B scripts/tests/test_demo.py
PYTHONPATH=scripts/tests/evaluation python3 -B -m unittest discover -s scripts/
    tests/evaluation -p test_perf03.py
./scripts/test.sh
```

```
sudo ./scripts/tests/test-e2e.sh
sudo ./scripts/tests/test-evaluation.sh E2E-18 E2E-19
```

Die ersten drei Befehle sind lokale Hilfsprüfungen. Die vollständige Prüfkette, Rustkomponenten mit Linux-APIs sowie die privilegierten Tests benötigen das dedizierte Linux-Zielsystem. E2E-01 bis E2E-19 prüfen die reale Runtime. COMP-, PERF-, LOAD-, STAB-, CHAR- und RACE-Szenarien untersuchen weitere Komponenten-, Mess- und Grenzfälle. Die vollständige Prüfkette enthält außerdem `scripts/tests/test_demo_live.py`: Der Test führt die automatische Demo mit realem tmux und Kernel-Enforcement unter einem Pseudoterminal aus und archiviert Report, Terminalmitschnitt und Demo-Fixture unter `test-results/`. Der privilegierte Live-Test ist kein Ersatz für lokale Demo-Regressionsprüfungen. Die Suite vor dem Start von fremden Runtime-Prozessen und vorhandenen Test-Pins freihalten; Suiten nicht parallel starten.

8.5 Erweiterungsschritte

1. Fachliche Semantik und Fehlerverhalten festlegen. Für zusätzliche Syntax Parser und Validierung in `tails-pdp-policy-loader/src/policy_source.rs` oder `tails-pdp-attribute-loader/src/stream_attributes.rs` anpassen.

2. Gemeinsame Repräsentation und reine Evaluierung in `tails-pdp-common/src/lib.rs` erweitern. Bestehende Map-Layouts und Enumwerte beachten; bei inkompatiblen Änderungen Migration und Pin-Bereinigung planen. Kapazitäten nicht nur im Parser, sondern auch in Maps und Prüfungen ändern.

3. Kernel- und Userspace-Auswertung auf dieselbe Semantik bringen. Fehlende Attribute, Typkonflikte, Wildcards und Zeitgrenzen ausdrücklich prüfen. Für einen neuen Hook zusätzlich Requestgewinnung, Programme, Maps, Lade-/Attach-Listen, Administration und Nachbewertung entwerfen.

4. Positive und negative Parser-/Komponententests sowie passende privilegierte Szenarien ergänzen. Dokumentation und Beispiele aktualisieren; erst danach Build, Clippy, Kernel-Verifier und reale Durchsetzung prüfen.

Die Wiederaufnahme einer Socket-Bind-Aktion ist eine neue Funktion und kein Umschalten eines vorhandenen Flags. Die vorherigen Implementierungsreste sind entfernt.

9 Grenzen und Nachweiszuordnung

Der Prototyp ist keine vollständige SAPL-Implementierung. Der Remote-FD-Entzug ist nur für x86_64-Linux implementiert und benötigt geeignete ptrace-Rechte. Bereits erzeugte Speicherabbildungen und konkurrierende FD-/Prozessoperationen sind nicht durch einen atomaren globalen Entzug abgedeckt. Ein bestandener Race-Test beweist keine Race-Freiheit. Dateiidentität und Pfadname dürfen nicht gleichgesetzt werden. Mehrdatei-Veröffentlichungen und Policy-/Attributaktivierung bilden keine gemeinsame

Transaktion. Eine einzelne Generationsumschaltung belegt keine unveränderte Lesesicht bei wiederholter Bankverwendung durch parallele Leser. Der synchrone Userspace-Scan und blockierendes Warten beim `ptrace`-Entzug können andere Zweige desselben Runtime-Tasks verzögern. Für das Warten besteht keine ausdrückliche Zeitbegrenzung; eine feste maximale Aktualisierungslatenz wird nicht zugesichert. Fehlgeschlagene Entzüge werden nicht ohne neuen Auslöser wiederholt.

Das Archiv `eval-tails-pdp/` enthält den Lauf vom 10. Oktober 2026 zum protokollierten Commit `a9fd6c0` mit dokumentiertem Arbeitsbaum. Seine Dateien sind über `eval-tails-pdp/SHA256SUMS` prüfbar. Bereinigungen nach diesem Lauf und dieses Handbuch sind kein neuer vollständiger Systemtestnachweis. Für die Abgabe den endgültigen Quellstand, Binaries und Nachweise eindeutig zuordnen.

10 Ausgabe des Handbuchs

Diese LaTeX-Datei benötigt keine eingebundenen Thesis-Dateien, Grafiken oder Literaturdaten. Der gemeinsame Build benötigt die Quellen unter `../thesis/` sowie LaTeX, `latexmk` und Biber. Aus der Repository-Wurzel:

```
bash scripts/build-thesis.sh
```

Die lokale Laufkonfiguration `thesis` verwendet denselben Helfer. Er baut Thesis und beide Handbücher und kopiert alle drei PDFs erst nach erfolgreicher Kompilierung nach `thesis/final/`. Dieses Handbuch liegt dort als `Christian_Koch_Programmierhandbuch_tails-pdp.pdf`. Die bearbeitbare Quelle verbleibt für die Abgabe im vorgesehenen Thesis-/source-Paket. Eine eigenständige Kompilierung dieses Handbuchs bleibt möglich.